

Отчет по лабораторной работе №7
«Патерн «Итератор», патерн «Фабричный метод» и средства рефлексии »

Выполнила: Качкуркина Арина Валерьевна

Группа: 6204-010302D

Задание 1

Я добавила в интерфейс TabulatedFunction.java:

```
package functions;
```

```
import java.util.Iterator;
```

```
public interface TabulatedFunction extends Function,  
Iterable<FunctionPoint>, Cloneable {
```

Добавлен итератор в ArrayTabulatedFunction.java:

```
//метод возвращает объект итератора для переб. точек
@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private int index = 0;

        //метод проверяет, есть ли след. элемент для итерации
        @Override
        public boolean hasNext() {
            return index < pointsCount; //проверим, есть ли след. элемент
        }

        //метод возвращает след. точку функции
        @Override
        public FunctionPoint next() {
            if (!hasNext()) { //если след. элемента нет -исключение
                throw new NoSuchElementException("Нет следующего элемента");
            }
            return new FunctionPoint(points[index++]); //возвр. копию точки
        }

        @Override
        public void remove() {
            throw new UnsupportedOperationException("Удаление не поддерживается");
        }
    };
}

//добавляем после итератора
public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory {
    @Override //через конструктор с границам и кол-ом точек
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override // через конструктор с границами и масс. Y
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }

    @Override //через конструктор с массивом точек
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new ArrayTabulatedFunction(points);
    }
}
```

Добавлен итератор в LinkedListTabulatedFunction.java:

```

//метод возвращает объект итератора для перебора
@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private FunctionNode currentNode = head.next;//тек. узел
        private int currentIndex = 0; //тек. индекс для проверки границ

        @Override
        public boolean hasNext() {
            return currentIndex < pointsCount;
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) { //если след. элемента нет - исключение
                throw new NoSuchElementException("Нет следующего элемента");
            }
            //создаем копи. точки из текущего. узла
            FunctionPoint point = new FunctionPoint(currentNode.point);
            currentNode = currentNode.next;//след. узел
            currentIndex++; //увеличиваем счетчик
            return point;
        }

        @Override
        public void remove() {
            throw new UnsupportedOperationException("Удаление не поддерживается");
        }
    };
}
//фабрика
public static class LinkedListTabulatedFunctionFactory implements TabulatedFunctionFactory {
    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new LinkedListTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new LinkedListTabulatedFunction(leftX, rightX, values);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new LinkedListTabulatedFunction(points);
    }
}

```

Результат выполнения программы:

Тестирование итераторов

ArrayTabulatedFunction:

(0.0; 0.0)

(2.0; 0.0)

(4.0; 0.0)

(6.0; 0.0)

(8.0; 0.0)

LinkedListTabulatedFunction:

(0.0; 0.0)

(2.0; 0.0)

(4.0; 0.0)

(6.0; 0.0)

(8.0; 0.0)

Явный итератор:

(0.0; 0.0)

(2.0; 0.0)

(4.0; 0.0)

(6.0; 0.0)

(8.0; 0.0)

Задание 2

Создала интерфейс TabulatedFunctionFactory.java:

```
package functions;

public interface TabulatedFunctionFactory {
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount); //создает ф-ию по границам и кол-ву точек
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values); //создает ф-ию по границам и массиву Y
    TabulatedFunction createTabulatedFunction(FunctionPoint[] points); //создает ф-ию по массиву точек
}
```

Фабрика в ArrayTabulatedFunction.java:

```
//добавляем после итератора
public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory {
    @Override //через конструктор с границами и кол-ом точек
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override // через конструктор с границами и масс. Y
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }

    @Override //через конструктор с массивом точек
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new ArrayTabulatedFunction(points);
    }
}
```

Фабрика в LinkedListTabulatedFunction.java:

```
//фабрика
public static class LinkedListTabulatedFunctionFactory implements TabulatedFunctionFactory {
    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new LinkedListTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new LinkedListTabulatedFunction(leftX, rightX, values);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new LinkedListTabulatedFunction(points);
    }
}
```

Активация Windows

Я изменила класс TabulatedFunctions.java:

//фабрика по умолчанию (ArrayTabulatedFunction)

```
private static TabulatedFunctionFactory factory = new
ArrayTabulatedFunction.ArrayTabulatedFunctionFactory();
```

//метод для смены фабрики

```
public static void setTabulatedFunctionFactory(TabulatedFunctionFactory factory) {
    TabulatedFunctions.factory = factory;
}
```

//методы создания через фабрику

```
public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, int
pointsCount) {
    return factory.createTabulatedFunction(leftX, rightX, pointsCount);
}
```

```
public static TabulatedFunction createTabulatedFunction(double leftX, double rightX,
double[] values) {
    return factory.createTabulatedFunction(leftX, rightX, values);
}
```

```
public static TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
```

return factory.createTabulatedFunction(points);

Результат выполнения программы:

Тестирование фабрик

Фабрика по умолчанию (Array):

Функция: {(0.0; 1.0), (0.7853981633974483; 0.7071067811865476), (1.5707963267948966; 6.123233995736766E-17), (2.356194490192345; -0.7071067811865475), (3.141592653589793; -1.0)}

Установка LinkedList фабрики:

Функция: {(0.0; 1.0), (0.7853981633974483; 0.7071067811865476), (1.5707963267948966; 6.123233995736766E-17), (2.356194490192345; -0.7071067811865475), (3.141592653589793; -1.0)}

Возврат к Array фабрике:

Функция: {(0.0; 1.0), (0.7853981633974483; 0.7071067811865476), (1.5707963267948966; 6.123233995736766E-17), (2.356194490192345; -0.7071067811865475), (3.141592653589793; -1.0)}

Задание 3

Реализовала три перегруженных метода createTabulatedFunction():

```
//методы создания через рефлексию
public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> functionClass, double leftX, double rightX, int pointsCount) {
    try { //ищем конструктор с параметрами (double, double, int)
        Constructor<? extends TabulatedFunction> constructor = functionClass.getConstructor(double.class, double.class, int.class);
        return constructor.newInstance(leftX, rightX, pointsCount); //создаем объект
    } catch (Exception e) {
        throw new IllegalArgumentException("Ошибка создания функции", e); //исключение IllegalArgumentException
    }
}

public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> functionClass, double leftX, double rightX, double[] values) {
    try {
        Constructor<? extends TabulatedFunction> constructor = functionClass.getConstructor(double.class, double.class, double[].class);
        return constructor.newInstance(leftX, rightX, values);
    } catch (Exception e) {
        throw new IllegalArgumentException("Ошибка создания функции", e);
    }
}

public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> functionClass, FunctionPoint[] points) {
    try {
        Constructor<? extends TabulatedFunction> constructor = functionClass.getConstructor(FunctionPoint[].class);
        return constructor.newInstance((Object) points);
    } catch (Exception e) {
        throw new IllegalArgumentException("Ошибка создания функции", e);
    }
}

//табулирование через рефлексию (задание 3)
public static TabulatedFunction tabulate(Class<? extends TabulatedFunction> functionClass, Function function, double leftX, double rightX, int pointsCount) {
    if (leftX < function.getLeftDomainBorder() || rightX > function.getRightDomainBorder()) {
        throw new IllegalArgumentException("Границы табулирования выходят за область определения");
    }

    if (pointsCount < 2) {
        throw new IllegalArgumentException("Количество точек должно быть не менее 2");
    }

    //создаем массив значений Y
    double[] values = new double[pointsCount];
    double step = (rightX - leftX) / (pointsCount - 1);
    //табулируем ф-ию
    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        values[i] = function.getFunctionValue(x);
    }
    //создаем ф-ию
    return createTabulatedFunction(functionClass, leftX, rightX, values);
}
```

Акт

Результат выполнения программы:

Рефлексия

Тестирование рефлексии

1. Создание ArrayTabulatedFunction через рефлексию:

Класс: ArrayTabulatedFunction

Значения: {(0.0; 0.0), (2.0; 0.0), (4.0; 0.0), (6.0; 0.0), (8.0; 0.0)}

2. Создание ArrayTabulatedFunction через рефлексию:

Класс: ArrayTabulatedFunction

Значения: {(0.0; 0.0), (8.0; 8.0)}

3. Создание через рефлекссию из массива точек:

Класс: LinkedListTabulatedFunction

Значения: {(0.0; 0.0), (8.0; 64.0)}

4. Табулирование с рефлексией:

Класс: LinkedListTabulatedFunction

Значения: {(0.0; 0.0), (0.3141592653589793; 0.3090169943749474), (0.6283185307179586; 0.5877852522924731), (0.9424777960769379; 0.8090169943749475), (1.2566370614359172; 0.9510565162951535), (1.5707963267948966; 1.0), (1.8849555921538759; 0.9510565162951536), (2.199114857512855; 0.8090169943749475), (2.5132741228718345; 0.5877852522924732), (2.827433388230814; 0.3090169943749475), (3.141592653589793; 1.2246467991473532E-16)}